Juan Cisneros jc222256
Sarah Rincon sr56495
Juan Arellano jma6353

Section 1: Introduction & Model Selection

This project models a 2024 Toyota Corolla quarter-car suspension system after the vehicle hits a small speed bump. A car suspension can be modeled as a spring-mass-damper system because the vehicle mass moves vertically, the spring provides a restoring force, and the shock absorber provides damping.
The governing second-order ODE is:

$$m\frac{d^2y}{dt^2} + c\frac{dy}{dt} + ky = 0$$

where y(t)y(t)y(t) is the vertical displacement from equilibrium.

To solve this numerically, the second-order ODE is converted into a system of first-order ODEs. Let:

$$y_1 = y$$
$$y_2 = \frac{dy}{dt}$$

In this model, m is the quarter-car mass, c is the damping coefficient, k is the spring constant, y_1 is displacement, and y_2 is velocity.

Section 2: Parameter Research & Justification

Parameter	**Value**	**Units**	**Source/ Justification**
Vehicle mass	1340	kg	2024 Toyota Corolla curb weight is about 2955 lb, converted to kg
Quarter- car mass (m)	335	kg	One-fourth of total vehicle mass
Damping coefficient (c)	545.5	Ns/m	Published quarter-car suspension model
Spring constant (k)	20000	N/m	Published quarter-car suspension model
Initial displacement (y(0)	0.05	m	Represents 5 cm suspension compression from a small bump
Initial velocity (y'(0))	0	m/s	Suspension is released from rest

The total vehicle mass was divided by four because this project uses a quarter-car suspension model, which represents one wheel and suspension corner. The spring constant and damping coefficient were chosen from published quarter-car suspension values because exact manufacturer suspension constants are usually not publicly listed.

The initial condition $y(0)=0.05$ m represents the suspension being compressed upward by 5 cm after hitting a small speed bump. The initial velocity $y'(0)=0$ m/s assumes the suspension starts from rest immediately after compression.

Final model:

$$335y'' + 545.5y' + 20000y = 0$$

References:

Toyota Corolla curb weight:
https://www.edmunds.com/toyota/corolla/2024/features-specs/

Quarter-car suspension parameters:
https://www.mechanics-industry.org/articles/meca/full_html/2019/06/mi180235/mi180235.html

Section 3: Numerical Methods Implementation

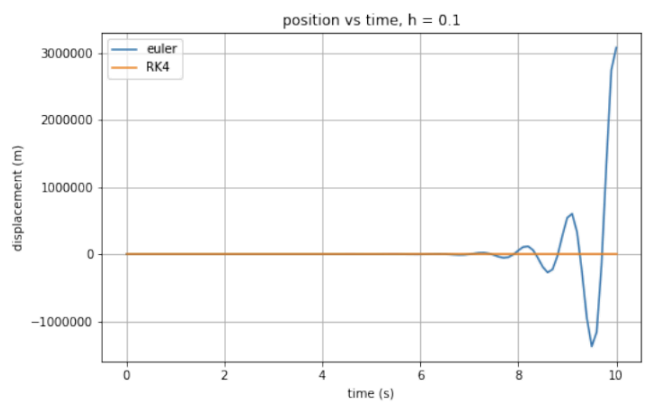
```python
import numpy as np
import matplotlib.pyplot as plt

# parameters
m = 335
c = 545.5
k = 20000

# initial conditions
y0 = 0.05     #initial displacement
v0 = 0.0      #initial velocity

#time settings
t0 = 0
tf = 10

def system(t, Y):
# converts the second-order ODE into a system of two first-order ODEs
# y[0] = displacement
# y[1] = velocity
    y = Y[0]
    v = Y[1]

    dydt = v
    dvdt = -(c/m)*v - (k/m)*y

    return np.array([dydt, dvdt])


def euler_method(h):

#forward euler method for solving the suspension system
    t_values = np.arange(t0, tf + h, h)
    Y = np.zeros((len(t_values), 2))

    Y[0] = [y0, v0]
```

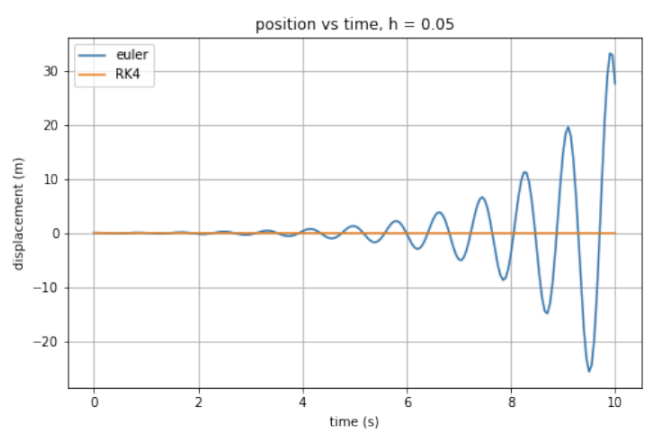
```python
    for i in range(len(t_values) - 1):
        Y[i+1] = Y[i] + h * system(t_values[i], Y[i])

    return t_values, Y


def rk4_method(h):
#fourth order Runge-Kutta method for solving the suspension system
    t_values = np.arange(t0, tf + h, h)
    Y = np.zeros((len(t_values), 2))

    Y[0] = [y0, v0]

    for i in range(len(t_values) - 1):
        t = t_values[i]

        k1 = system(t, Y[i])
        k2 = system(t + h/2, Y[i] + (h/2)*k1)
        k3 = system(t + h/2, Y[i] + (h/2)*k2)
        k4 = system(t + h, Y[i] + h*k3)

        Y[i+1] = Y[i] + (h/6)*(k1 + 2*k2 + 2*k3 + k4)

    return t_values, Y
```

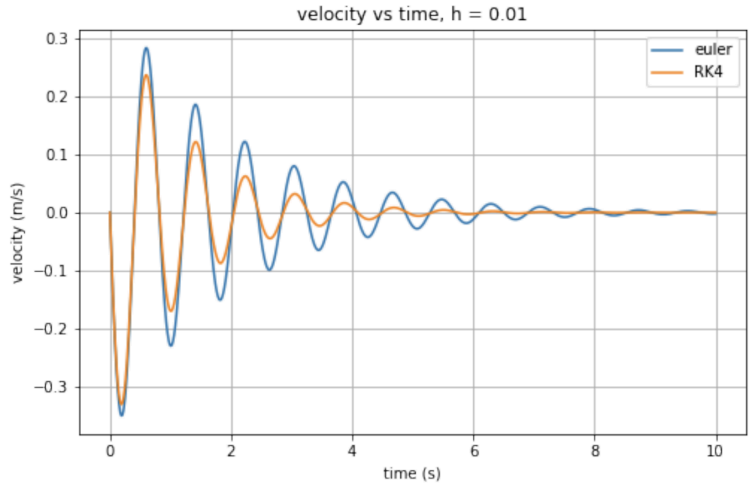
The Euler method estimates the next point using only the slope at the beginning of the time step. RK4 uses four slope estimates per time step, making it more accurate and stable than Euler's method.

Section 4: Solutions and Comparison

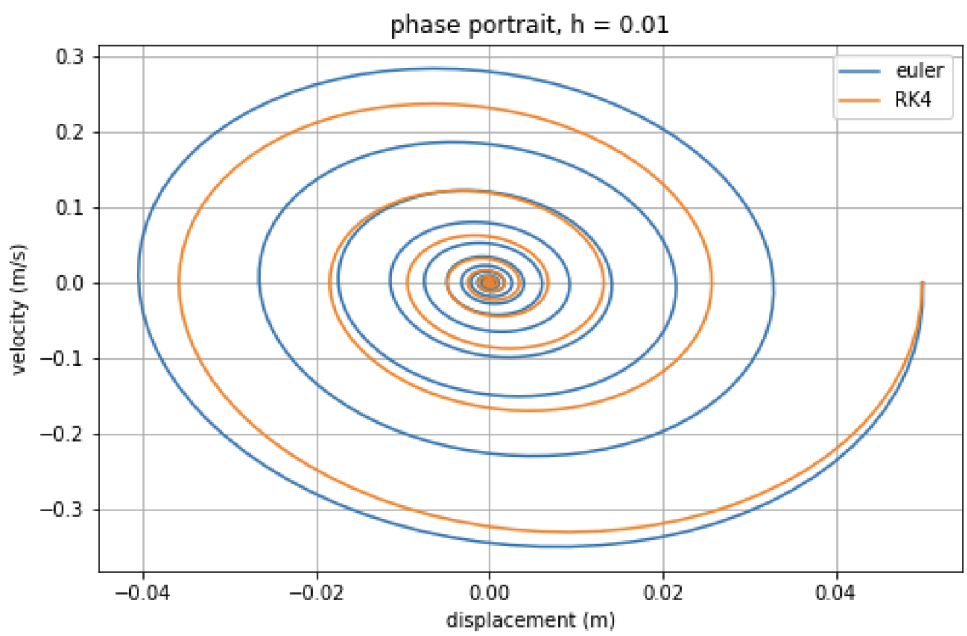
```
h_values = [0.1, 0.05, 0.01]

for h in h_values:
    t_euler, Y_euler = euler_method(h)
    t_rk4, Y_rk4 = rk4_method(h)

    plt.figure(figsize=(8,5))
    plt.plot(t_euler, Y_euler[:,0], label="euler")
    plt.plot(t_rk4, Y_rk4[:,0], label="RK4")
    plt.xlabel("time (s)")
    plt.ylabel("displacement (m)")
    plt.title("position vs time, h = " + str(h))
    plt.legend()
    plt.grid(True)
    plt.show()
```







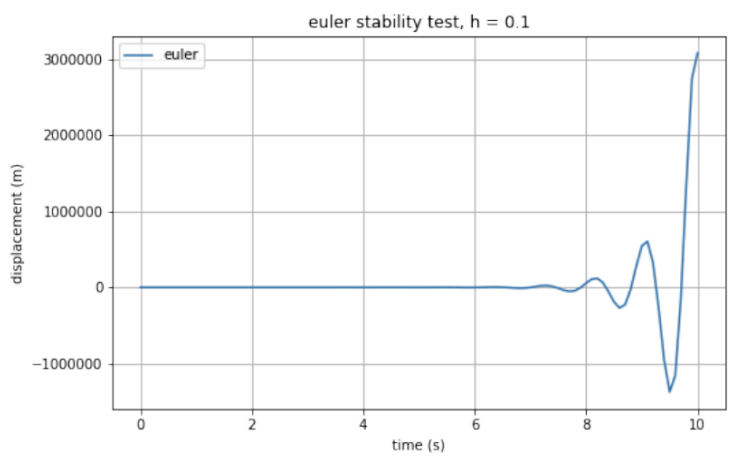

```
h = 0.01

t_euler, Y_euler = euler_method(h)
t_rk4, Y_rk4 = rk4_method(h)

plt.figure(figsize=(8,5))
plt.plot(Y_euler[:,0], Y_euler[:,1], label="euler")
plt.plot(Y_rk4[:,0], Y_rk4[:,1], label="RK4")
plt.xlabel("displacement (m)")
plt.ylabel("velocity (m/s)")
plt.title("phase portrait, h = 0.01")
plt.legend()
plt.grid(True)
plt.show()
```

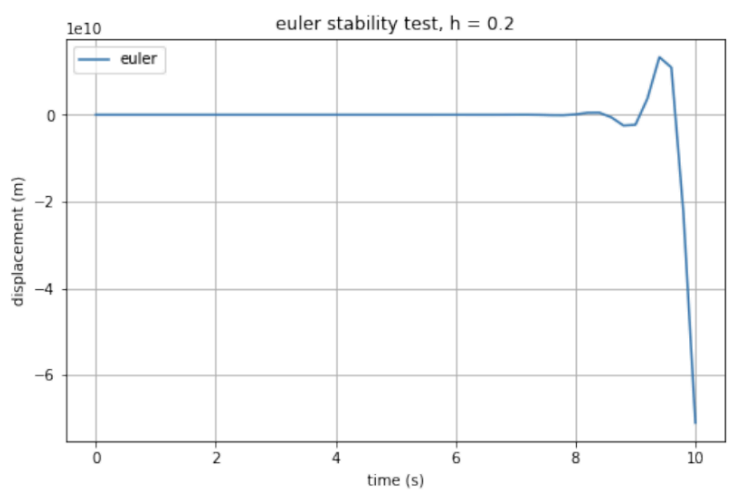


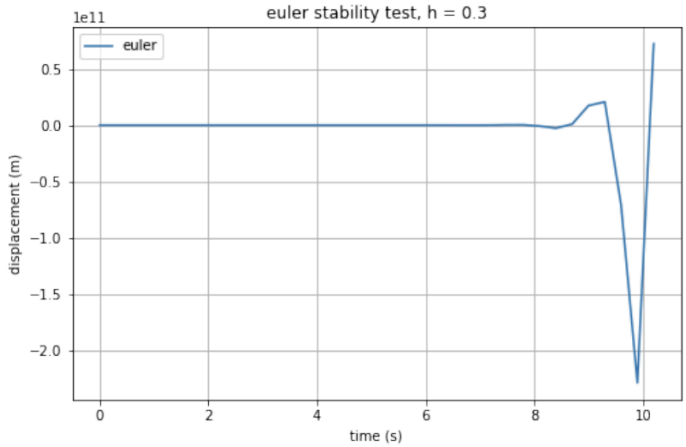
As the step size decreased, both Euler and RK4 gave smoother and more accurate results. For larger step sizes, Euler showed more error and drift from the expected damped oscillation. RK4 stayed smoother and followed the physical behavior better. The phase portrait also shows the system spiraling toward equilibrium, which makes sense because damping removes energy from the suspension.

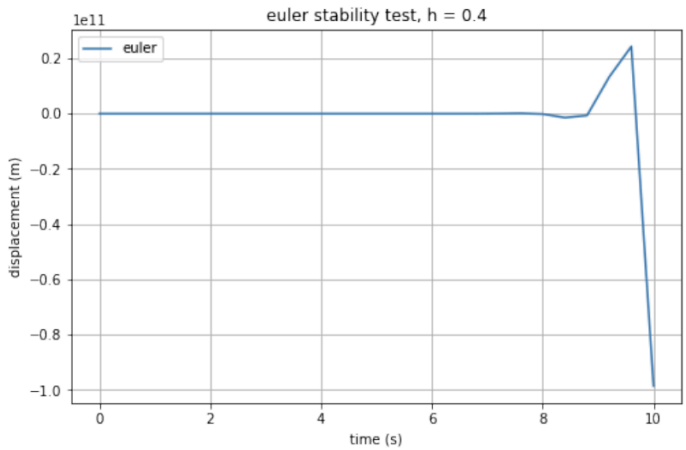
Section 5

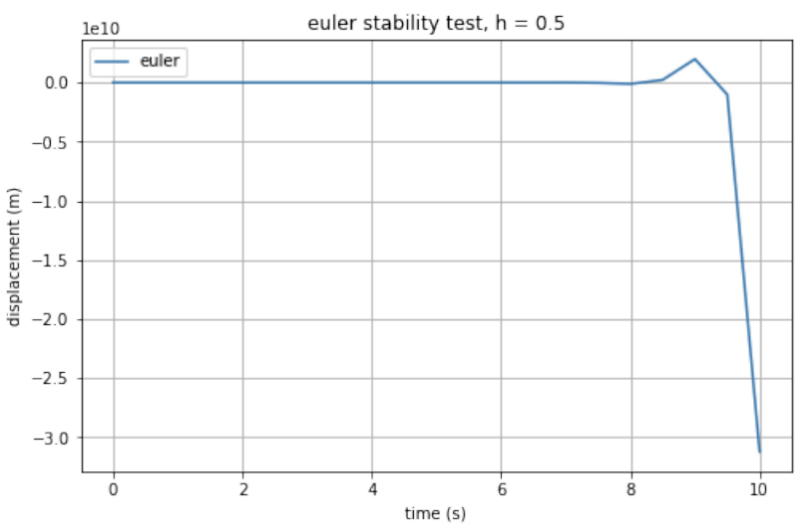
```
large_h_values = [0.1, 0.2, 0.3, 0.4, 0.5]

for h in large_h_values:
    t_euler, Y_euler = euler_method(h)

    plt.figure(figsize=(8,5))
    plt.plot(t_euler, Y_euler[:,0], label="euler")
    plt.xlabel("time (s)")
    plt.ylabel("displacement (m)")
    plt.title("euler stability test, h = " + str(h))
    plt.legend()
    plt.grid(True)
    plt.show()
```





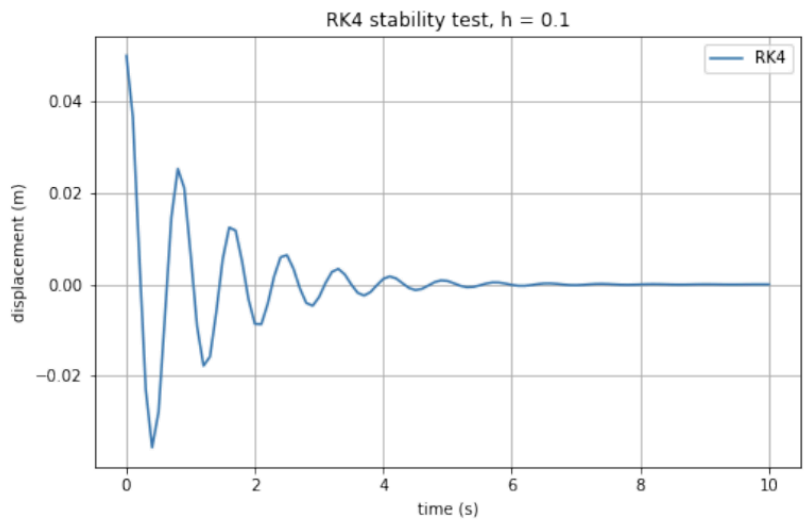

euler stability test, h = 0.3
1e11
euler
0.5
0.0
−0.5
−1.0
−1.5
−2.0
displacement (m)
0
2
4
6
8
10
time (s)

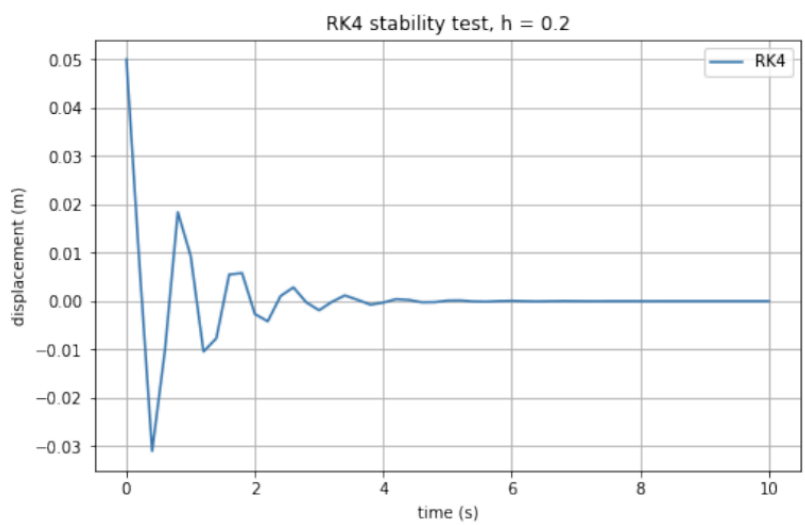

euler stability test, h = 0.4
1e11
euler
0.2
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
displacement (m)
0
2
4
6
8
10
time (s)

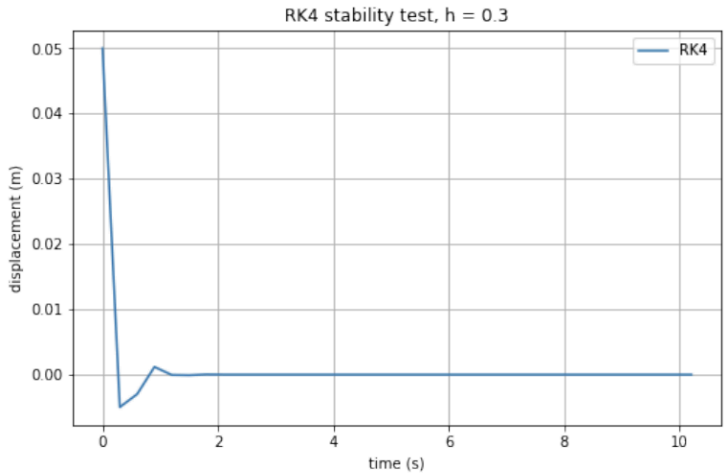

euler stability test, h = 0.5
1e10
euler
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
displacement (m)
0
2
4
6
8
10
time (s)

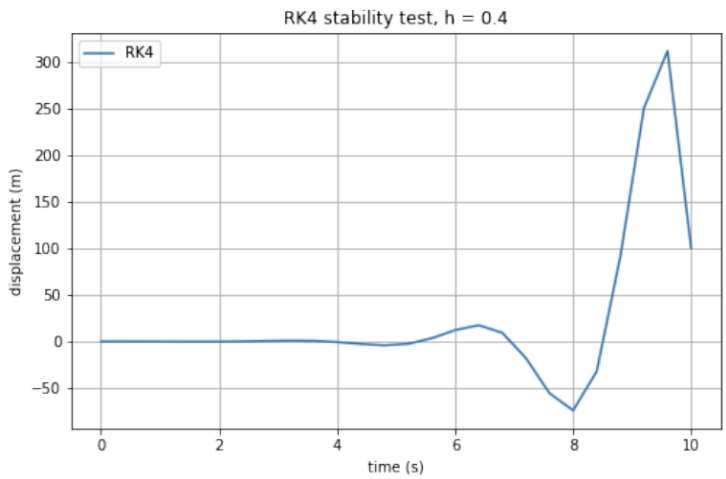
RK4

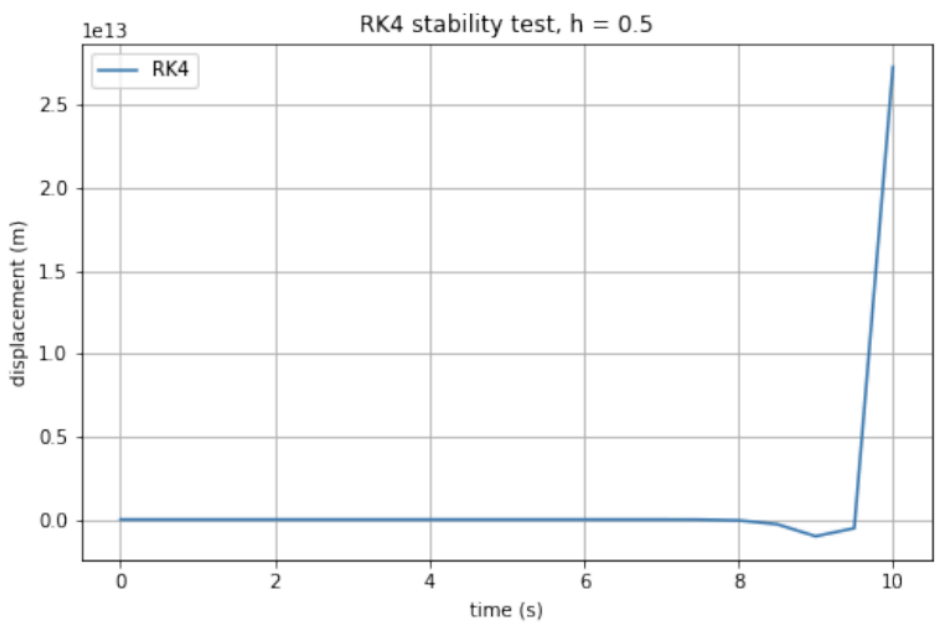
```
for h in large_h_values:
    t_rk4, Y_rk4 = rk4_method(h)

    plt.figure(figsize=(8,5))
    plt.plot(t_rk4, Y_rk4[:,0], label="RK4")
    plt.xlabel("time (s)")
    plt.ylabel("displacement (m)")
    plt.title("RK4 stability test, h = " + str(h))
    plt.legend()
    plt.grid(True)
    plt.show()
```






RK4 stability test, h = 0.3
RK4
displacement (m)
time (s)


RK4 stability test, h = 0.4
RK4
displacement (m)
time (s)


RK4 stability test, h = 0.5
1e13
RK4
displacement (m)
time (s)

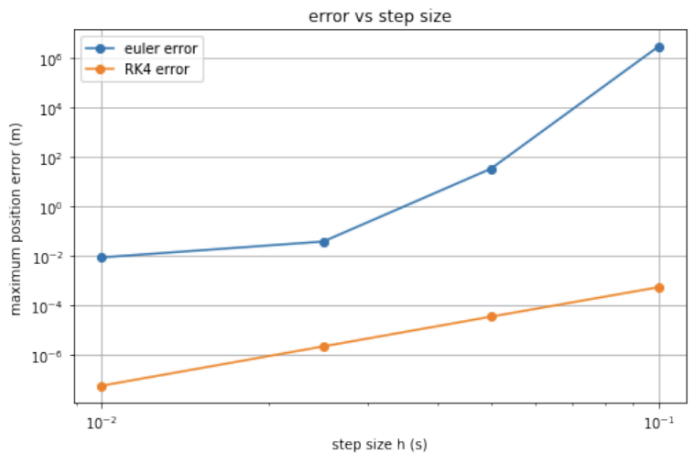
For small step sizes, both methods show the expected damped oscillation. The displacement gradually decreases over time because the damper removes energy from the system. When the step size becomes too large, Euler's method can become unstable and may show oscillations growing instead of decaying. This is non-physical because a damped suspension should not gain energy.

RK4 remains stable for larger step sizes than Euler. This shows that RK4 has better stability behavior and is more reliable for this type of engineering simulation.

Error Analysis

```
#use RK4 with very small step size
h_ref = 0.001
t_ref, Y_ref = rk4_method(h_ref)

def get_reference_values(t_values):
    ref_position = np.interp(t_values, t_ref, Y_ref[:,0])
    return ref_position

h_error_values = [0.1, 0.05, 0.025, 0.01]

euler_errors = []
rk4_errors = []

for h in h_error_values:
    t_euler, Y_euler = euler_method(h)
    t_rk4, Y_rk4 = rk4_method(h)

    ref_euler = get_reference_values(t_euler)
    ref_rk4 = get_reference_values(t_rk4)

    euler_error = np.max(np.abs(Y_euler[:,0] - ref_euler))
```

```
    rk4_error = np.max(np.abs(Y_rk4[:,0] - ref_rk4))

    euler_errors.append(euler_error)
    rk4_errors.append(rk4_error)

plt.figure(figsize=(8,5))
plt.loglog(h_error_values, euler_errors, marker="o", label="euler error")
plt.loglog(h_error_values, rk4_errors, marker="o", label="RK4 error")
plt.xlabel("step size h (s)")
plt.ylabel("maximum position error (m)")
plt.title("error vs step size")
plt.legend()
plt.grid(True)
plt.show()
```




```
#estimate slopes of plots
euler_slope = np.polyfit(np.log(h_error_values), np.log(euler_errors), 1)[0]
rk4_slope = np.polyfit(np.log(h_error_values), np.log(rk4_errors), 1)[0]

print("euler convergence slope=", euler_slope)
print("RK4 convergence slope=", rk4_slope)
```

```
euler convergence slope= 8.441653025616406
RK4 convergence slope= 3.9998034299565624
```

The error decreased as the step size decreased. Euler's method had a convergence rate close to first order, meaning the error is proportional to h. RK4 had a much faster decrease in error, close to fourth order. This means RK4 becomes much more accurate than Euler when the step size is reduced.

Physically, instability means the simulation is adding artificial energy into the suspension. In a real damped suspension, the motion should decay over time. If the numerical solution grows instead, then the method is not accurately representing the real system.

Section 6: Conclusion

RK4 performed better than Euler's method for this suspension problem. It produced smoother, more accurate, and more stable results, especially when larger step sizes were used. Euler's

method was easier to implement and required less computation per step, but it became inaccurate and unstable much faster.

The tradeoff is that RK4 requires more calculations at each step because it computes four slopes instead of one. However, the extra cost is worth it because RK4 can use larger step sizes while still giving accurate results. For practical engineering simulations, RK4 is the better method for this type of damped suspension model.

This project showed that numerical method choice matters. A method that is too simple or uses a step size that is too large can produce non-physical results, even when the model itself is correct.

Citation:

AI Assistance: Used ChatGPT for minor debugging and formatting help in Python.

Prompt was focused on fixing syntax and improving code readability, not generating full solutions.